Building atlas-free cortical asymmetry modules in Python

CorticalFields can implement spectral and transport-based asymmetry analysis using well-supported Python libraries, with **pyFM natively accepting point clouds** (no mesh reconstruction needed), **POT's sliced Wasserstein distance completing 96k pairwise comparisons in ~1 hour on 16 cores**, and robust_laplacian providing guaranteed positive semi-definite Laplacians for ~150k-vertex cortical surfaces. The entire pipeline—eigendecomposition, pairwise distances, and statistical inference—runs end-to-end in approximately 2–5 hours on a 16-core workstation for 439 subjects. Below is every API signature, parameter choice, and integration detail needed to build the three modules.

Module 1A: extracting cortical point clouds from FreeSurfer surfaces

Use FreeSurfer pial vertices directly—not marching cubes. The pial surface files (`lh.pial`, `rh.pial`) already contain topology-corrected, sub-voxel-accurate point clouds at ~150k vertices. Marching cubes on a binary NIfTI mask produces staircase artifacts, non-manifold edges, and often >500k vertices that must be decimated anyway.

```python
import nibabel.freesurfer as fs
import numpy as np

coords, faces = fs.read_geometry('/subjects/sub-01/surf/lh.pial')
# coords: (N, 3) float64 — vertices in tkRAS coordinates (FreeSurfer-specifi
# faces:  (F, 3) int32   — triangle indices, F ≈ 2N
```

Critical coordinate system issue: FreeSurfer pial vertices live in "tkRAS" (tkregister RAS), not scanner RAS. (nipy +2) To overlay on NIfTI volumes or combine with other modalities, convert explicitly:

```python
import nibabel as nib

t1 = nib.load('mri/T1.mgz')
vox2ras = t1.header.get_vox2ras()
vox2ras_tkr = t1.header.get_vox2ras_tkr()
tkr2scanner = vox2ras @ np.linalg.inv(vox2ras_tkr)

coords_hom = np.hstack([coords, np.ones((coords.shape[0], 1))])
coords_ras = (tkr2scanner @ coords_hom.T).T[:, :3]
```

For projects that lack FreeSurfer reconstructions and must extract from NIfTI volumes, marching cubes works as a fallback. The key detail: `skimage.measure.marching_cubes(volume, level=0.5)` returns vertices in **voxel coordinates**— scikit-image apply the NIfTI affine via homogeneous multiplication (`affine @ [i,j,k,1]`) to get scanner RAS in millimeters.

Open3D normal estimation for ~150k-vertex cortical surfaces should use `radius=3.0` mm and `max_nn=30`. Typical inter-vertex spacing on FreeSurfer pial is **0.7–1.0 mm**, so a 3 mm radius captures 20–30 neighbors per point—enough for stable PCA-based normals without over-smoothing sulcal curvature: Open3D

```python
import open3d as o3d

pcd = o3d.geometry.PointCloud()
pcd.points = o3d.utility.Vector3dVector(coords)
pcd.estimate_normals(
    search_param=o3d.geometry.KDTreeSearchParamHybrid(radius=3.0, max_nn=30)
)
pcd.orient_normals_consistent_tangent_plane(k=15)
```

Open3D's `compute_point_cloud_distance(target)` returns an `(N,)` array of per-point nearest-neighbor Euclidean distances from source to target—useful for Chamfer distance (`mean(d_src→tgt) + mean(d_tgt→src)`). Open3D ICP registration between mirrored hemispheres uses `o3d.pipelines.registration.registration_icp()` with `TransformationEstimationPointToPlane()` Open3D and `max_correspondence_distance=5.0` mm as an initial generous threshold. Open3D

Module 1B: the Laplace-Beltrami operator via robust_laplacian

The `robust_laplacian` library (Nick Sharp, geometry-central C++ backend via pybind11) PyPI is the correct choice. Its single function call returns a guaranteed **positive semi-definite** Laplacian—diagonal entries positive, off-diagonal negative, all edge weights positive— PyPI GitHub unlike libigl's `cotmatrix` which uses the opposite sign convention.

```python
import robust_laplacian
import scipy.sparse.linalg as sla

L, M = robust_laplacian.point_cloud_laplacian(
    coords,            # (N, 3) float64
    n_neighbors=30,    # k-NN for local Delaunay triangulation
    mollify_factor=1e-5  # intrinsic mollification; increase to 1e-3 for deg
)
# L: scipy.sparse.csc_matrix (N, N), positive semi-definite
# M: scipy.sparse.csc_matrix (N, N), diagonal lumped mass matrix
```

The generalized eigenvalue problem is `Lφ = λMφ` with **λ ≥ 0**. Use shift-invert mode (`sigma` near zero) for ARPACK to efficiently find the smallest eigenvalues:

```python
evals, evecs = sla.eigsh(L, k=100, M=M, sigma=1e-8)
# evals: (100,) non-negative, sorted ascending
# evecs: (N, 100) orthonormal w.r.t. M
```

Non-uniform density handling is implicit: the algorithm projects each point's k-NN neighborhood onto a local tangent plane, constructs an intrinsic Delaunay triangulation in 2D, then computes cotangent weights on those triangles. Dense regions (gyral crowns) produce small triangles with small mass entries; sparse regions (deep sulci) produce larger triangles. The mass matrix M encodes these area differences naturally. The `mollify_factor` parameter (default **1e-5**) adds a small perturbation that prevents floating-point degeneracy in near-collinear point configurations common in deep sulci—values up to **1e-3** are safe without geometric distortion. github

The function **does not accept pre-computed normals**. It computes tangent planes internally as part of the local triangulation.

Memory footprint for 150k points with k=30: the sparse Laplacian L has approximately **3M non-zero entries** (~15–20 per row, symmetric), consuming ~36 MB. With the mass matrix M (~1.2 MB) and internal working memory, peak usage is **~50–100 MB**. Computation takes **2–5 seconds** on the C++ backend.

Compared to manually building a k-NN graph with inverse-distance or cotangent weights, robust_laplacian's intrinsic Delaunay approach guarantees positive edge weights and PSD structure. Manual cotangent weights on obtuse triangles can go negative, breaking spectral methods downstream. For production cortical analysis, robust_laplacian is strictly preferred.

One known issue (GitHub #12): some users report CHOLMOD errors claiming L is not positive definite, likely from near-degenerate point configurations. (github) The fix is to increase `mollify_factor` to **1e-4** or **1e-3**. (GitHub)

Module 1C: functional maps with pyFM

pyFM works with point clouds directly. When `TriMesh` receives vertices without faces, it automatically sets `robust=True` and calls `robust_laplacian.point_cloud_laplacian()` internally. No mesh reconstruction is needed.

```python
from pyFM.mesh import TriMesh
from pyFM.functional import FunctionalMapping

# Point cloud mode (no faces)
mesh_lh = TriMesh(coords_lh)  # (N, 3) array; faces=None triggers robust_lap
mesh_lh.process(k=100, intrinsic=True, verbose=True)
# Computes: mesh_lh.eigenvalues (100,), mesh_lh.eigenvectors (N, 100)
#           mesh_lh.W (stiffness), mesh_lh.A (mass)

# With FreeSurfer mesh (preferred for higher-quality Laplacian)
mesh_lh = TriMesh(coords_lh, faces_lh, area_normalize=True, center=True)
mesh_lh.process(k=100, intrinsic=True)
```

Install via `pip install pyfmaps` (PyPI package name is `pyfmaps`, not `pyFM`). (GitHub)

The functional map C is computed by solving a regularized least-squares problem:

$$\min_C \|CA - B\|^2_F + \lambda_{lap} \cdot \|C\Lambda_1 - \Lambda_2 C\|^2_F$$

where A and B are spectral projections of descriptor functions, and Λ_1, Λ_2 are diagonal eigenvalue matrices. The Laplacian commutativity term enforces that C commutes with the Laplace-Beltrami operator—a necessary condition for isometries. Polytechnique

```python
model = FunctionalMapping(mesh_lh, mesh_rh_mirrored)
model.preprocess(n_ev=100, subsample_step=5, n_descr=100)
# Computes WKS descriptors by default
model.fit(optinit='zeros')  # or 'identity' or 'random'
```

Custom descriptors from CorticalFields' spectral.py can be passed by setting `model.descr1` and `model.descr2` directly—these are `(N, p)` arrays of `p` descriptor functions evaluated at each vertex. Project them into the spectral basis with `mesh.project(func)`, returning `(k,)` coefficients.

ZoomOut refinement is built in and is the recommended post-processing step:

```python
model.zoomout_refine(nit=15, step=1, subsample=3000, use_ANN=False)
# Starts at current k_init (from initial FM size)
# k_final = k_init + nit × step  (e.g., 20 + 15×1 = 35 → increase to 100 wit
```

For cortical surfaces, a practical configuration is `k_init=20`, then ZoomOut with `nit=80, step=1` to reach `k_final=100`. Each iteration converts C to a pointwise map via nearest-neighbor in spectral space, then re-estimates C at one higher spectral dimension. arXiv TheCVF With `subsample=3000`, each iteration runs in milliseconds.

For inter-hemispheric maps, you must preprocess the right hemisphere before computing the functional map:

```python
# 1. Mirror right hemisphere (negate x in RAS)
rh_coords_mirrored = rh_coords.copy()
rh_coords_mirrored[:, 0] *= -1

# 2. Fix face winding (reflection reverses orientation)
rh_faces_mirrored = rh_faces[:, [0, 2, 1]]  # swap columns 1 and 2

# 3. Build pyFM mesh from mirrored geometry
mesh_rh = TriMesh(rh_coords_mirrored, rh_faces_mirrored, area_normalize=True
mesh_rh.process(k=100, intrinsic=True)
```

Failing to reverse face winding after reflection causes the mesh to appear "inside-out," (github) producing incorrect cotangent weights and mass matrix entries. **pyFM does not handle this automatically**—it is geometry-agnostic.

The resulting C matrix for hemispheres that are near-symmetric will be **approximately diagonal with ±1 entries**. Even eigenfunctions get +1 (preserved under reflection), odd eigenfunctions get −1 (sign-flipped). This is not a permutation matrix—it is a signed diagonal matrix for perfect intrinsic symmetry.

Module 1D: asymmetry metrics derived from the C matrix

Off-diagonal energy is the primary asymmetry scalar:

```python
C = model.FM  # (k, k) functional map matrix
C_diag = np.diag(np.diag(C))
E_off = np.linalg.norm(C - C_diag, 'fro')
E_off_normalized = E_off / np.linalg.norm(C, 'fro')  # range [0, 1]
```

This formulation measures how far C deviates from diagonal—**zero for perfect bilateral symmetry**, increasing with asymmetry. Normalizing by `||C||_F` controls for overall map quality and eigenvector count.

Frequency-band decomposition partitions C into blocks that separate large-scale from fine-scale asymmetry:

```python
C_low = C[:20, :20]        # low-frequency: global shape asymmetry
C_mid = C[20:60, 20:60]    # mid-frequency: sulcal pattern differences
C_high = C[60:100, 60:100] # high-frequency: local cortical folding

E_low = np.linalg.norm(C_low - np.diag(np.diag(C_low)), 'fro')
E_high = np.linalg.norm(C_high - np.diag(np.diag(C_high)), 'fro')

# Off-diagonal BLOCKS capture frequency mixing (low-freq maps to high-freq)
C_cross = C[:20, 20:100]   # cross-frequency coupling
E_cross = np.linalg.norm(C_cross, 'fro')
```

The cross-frequency blocks `C[0:20, 20:100]` are especially informative—they reveal where low-frequency structure on one hemisphere maps to high-frequency structure on the other, a signature of asymmetric folding complexity.

SVD of C (`U, Σ, Vᵀ = np.linalg.svd(C)`) yields singular values that should all equal 1.0 for a perfect orthogonal map. Deviation of Σ from the all-ones vector quantifies non-isometric distortion. The dominant left/right singular vector pairs identify **primary asymmetry modes**—specific spectral patterns that most differ between hemispheres.

Group-level distance matrix from N subjects' C matrices:

```python
# Frobenius distance between C matrices
D = np.zeros((N, N))
for i in range(N):
    for j in range(i+1, N):
        D[i,j] = D[j,i] = np.linalg.norm(C_all[i] - C_all[j], 'fro')
```

Alternative: vectorize each C into a k^2-dimensional vector and use `scipy.spatial.distance.pdist` with Euclidean or correlation distance.

Module 2A: sliced Wasserstein distance via POT

The POT library's sliced Wasserstein function is the best choice for building pairwise distance matrices across hundreds of subjects—it is **fast, a true metric, and yields a provably positive definite kernel.** JMLR

```python
import ot

d = ot.sliced.sliced_wasserstein_distance(
    X_s,                # (n, d) ndarray — source point cloud
    X_t,                # (m, d) ndarray — target point cloud
    a=weights_s,        # (n,) weights, or None for uniform 1/n
    b=weights_t,        # (m,) weights, or None for uniform 1/m
    n_projections=200, # default is 50; 200 gives good precision
    p=2,                # Wasserstein-p exponent
    seed=42             # reproducibility
)
# Returns: scalar float64
```

Input arrays can have **d > 3**: append cortical thickness, curvature, or sulcal depth as extra dimensions to create morphometric point clouds in R^d. Each random projection slices this d-dimensional space onto a 1D line, sorts the projections, and computes the closed-form 1D Wasserstein distance. (Pythonot) Complexity is **O(n_projections × n log n)** with **O(nd)** memory—no quadratic cost matrix.

Weights (`a`, `b`) allow weighting each vertex by its associated area element from the mass matrix M. This is important for non-uniform point density: `a = np.diag(M.toarray()); a /= a.sum()`.

Benchmarks for 150k × 3 with 200 projections: each projection involves sorting 150k elements (~0.5 ms in NumPy), so 200 projections take approximately **0.2–2 seconds per pair on CPU**. For 96,141 pairwise comparisons (439 subjects), the total is ~14 hours serial or **~1 hour on 16 cores** via `joblib.Parallel(n_jobs=16, backend='loky')`.

The **sliced Wasserstein kernel** `K(x,y) = exp(-γ · SW²(x,y))` is provably positive definite for all γ > 0, proven by Kolouri et al. (CVPR 2016). The proof relies on SW^2 being a conditionally negative definite (Hilbertian) distance, with Schoenberg's theorem guaranteeing the Gaussian kernel on such distances is PD. (arXiv) (IEEE Xplore) This kernel can be passed directly to `KernelRidge(kernel="precomputed")`. (scikit-learn)

Module 2B: Sinkhorn divergence via GeomLoss

GeomLoss provides a differentiable, GPU-accelerated Sinkhorn divergence with multiscale acceleration. (Kernel-operations) The critical parameter choices for cortical surfaces:

```python
import torch
from geomloss import SamplesLoss

loss_fn = SamplesLoss(
    loss="sinkhorn",
    p=2,
    blur=5.0,       # σ in mm; cortical surfaces span ~200mm diameter
    scaling=0.5,    # ε-scaling ratio; 0.5 = default speed/accuracy tradeoff
    reach=None,     # None = balanced OT; set ~20mm for unbalanced
    debias=True,    # Sinkhorn divergence (unbiased), not OT_ε
    backend="online" # CRITICAL for 150k points — linear memory
)

x = torch.tensor(lh_coords, dtype=torch.float32).cuda()
y = torch.tensor(rh_coords_mirrored, dtype=torch.float32).cuda()
value = loss_fn(x, y)  # scalar tensor

# With weights:
alpha = torch.tensor(weights_lh, dtype=torch.float32).cuda()
beta = torch.tensor(weights_rh, dtype=torch.float32).cuda()
value = loss_fn(alpha, x, beta, y)
```

`blur` **selection**: the default (0.05) assumes data in [0,1]^d. Kernel-operations For cortical surfaces in native millimeters (diameter ~200 mm), use **blur = 1.0–10.0 mm**. A blur of 5 mm resolves gyral-scale structure; 1 mm captures sulcal detail but requires more Sinkhorn iterations.

Memory: the `"tensorized"` backend materializes an N×N cost matrix—at 150k points that is **90 GB, guaranteed OOM**. The `"online"` backend (requires PyKeOps + CUDA) uses **linear memory** and processes 150k points in ~0.3 seconds on a V100. Kernel-operations The `"multiscale"` backend is even faster (~0.07s at 100k) with O(N log N) time for 3D data. GeomLoss **can run on CPU** with the tensorized backend, but this is limited to ~10k points.

GeomLoss returns only the **divergence value** (or dual potentials if `potentials=True`), not an explicit transport plan. Kernel-operations For cortical asymmetry, the scalar divergence is sufficient.

Module 2C: FUGW for structure-preserving alignment

FUGW (Fused Unbalanced Gromov-Wasserstein) from INRIA's Parietal team jointly matches **functional features and anatomical geometry.** (arXiv) (ResearchGate) Install via `pip install fugw`. The API:

```python
from fugw.mappings import FUGWSparse

mapping = FUGWSparse(
    alpha=0.5,     # 0 = pure feature Wasserstein, 1 = pure geometry GW
    rho=1e-2,      # marginal relaxation (unbalanced parameter)
    eps=1e-4       # entropic regularization
)

mapping.fit(
    source_features=source_features,            # (n_contrasts, n_source) te
    target_features=target_features,            # (n_contrasts, n_target) te
    source_geometry_embeddings=source_lmds,     # (n_source, k) tensor
    target_geometry_embeddings=target_lmds,     # (n_target, k) tensor
    solver="sinkhorn",
    solver_params={"nits_bcd": 5, "tol_uot": 1e-10},
    source_sample=source_subsample_indices,     # ~1000 landmark vertices
    target_sample=target_subsample_indices,
    source_selection_radius=radius,
    target_selection_radius=radius,
)
```

FUGW outputs a **transport plan matrix P** plus per-iteration loss components (Wasserstein, Gromov-Wasserstein, marginal constraints). The `FUGWSparse` solver handles 150k vertices (Alexisthual) using a coarse-to-fine strategy: uniformly subsample ~1000 vertices, fit a coarse mapping, then refine with local neighborhoods. This takes **<10 minutes per pair on a V100.** (Alexisthual)

FUGW is not feasible for pairwise distance matrices. At 10 minutes per pair × 96k pairs = ~341 GPU-days. Use it for detailed vertex-level alignment on specific subject pairs, not as a batch distance metric. The FUGW objective value is also **non-convex** (depends on initialization), so it does not satisfy metric axioms. (Alexisthual) For distance matrices, use POT's sliced Wasserstein.

Module 3: statistical inference on distance matrices

MDMR exists as a Python implementation

The cleanest standalone implementation is `MDMR_fMRI` (Misaki et al., 2018, NeuroImage Clinical) at `github.com/mamisaki/MDMR_fMRI`. GitHub C-PAC also includes MDMR in `CPAC.cwas.mdmr`. Fcp-indi Fcp-indi The core algorithm in four steps:

```python
import numpy as np

# 1. Gower-center the squared distance matrix
n = D.shape[0]
H = np.eye(n) - np.ones((n, n)) / n
G = -0.5 * H @ (D ** 2) @ H

# 2. Hat matrix from design matrix X (with intercept + covariates)
Q, R = np.linalg.qr(X)
H_hat = Q @ Q.T

# 3. Pseudo-F statistic
m = X.shape[1] - 1  # degrees of freedom for variables of interest
numerator = np.trace(H_hat @ G @ H_hat) / m
denominator = np.trace((np.eye(n) - H_hat) @ G @ (np.eye(n) - H_hat)) / (n -
F_obs = numerator / denominator

# 4. Freedman-Lane permutation (for covariate control)
X_interest = X[:, 1:]   # columns of interest
Z = X[:, :1]            # nuisance (e.g., intercept + covariates)
H_Z = Z @ np.linalg.pinv(Z)
R_Z = np.eye(n) - H_Z

F_null = np.zeros(n_perms)
for p in range(n_perms):
    perm = np.random.permutation(n)
    Y_star = (H_Z + np.eye(n)[perm] @ R_Z)  # permuted projection
    # Recompute G from permuted data, then F
    ...
```

The **Freedman-Lane procedure** permutes residuals from the reduced model (covariates only), adds back the predicted values, then re-fits the full model. This correctly isolates the effect of interest while controlling for age, sex, and disease duration. **10,000 permutations** is publication-standard; C-PAC defaults to 15,000. Fcp-indi

hyppo handles precomputed distance matrices

```python
from hyppo.independence import Dcorr, Hsic

# Distance correlation with precomputed distance matrix
stat, pvalue = Dcorr(compute_distance=None).test(
    D_brain, D_behavior,
    reps=10000,
    auto=False,      # CRITICAL: auto=True uses chi2 approximation for n>20
    workers=-1       # all cores
)

# HSIC with precomputed kernel matrix
stat, pvalue = Hsic(compute_kernel=None).test(
    K_brain, K_behavior,
    reps=10000,
    auto=False,
    workers=-1
)
```

Setting `auto=False` is essential—the default `auto=True` bypasses permutation testing for n > 20, using a fast chi-squared approximation Neurodata that may be less appropriate for small neuroimaging samples.

Kernel Ridge Regression with precomputed kernels

```python
from sklearn.kernel_ridge import KernelRidge
from sklearn.model_selection import LeaveOneOut

# Distance → kernel via median heuristic
sigma = np.median(D[np.triu_indices_from(D, k=1)])
K = np.exp(-D**2 / (2 * sigma**2))

# LOO cross-validation (n=46)
loo = LeaveOneOut()
y_pred = np.zeros(len(y))
for train_idx, test_idx in loo.split(K):
    K_train = K[np.ix_(train_idx, train_idx)]   # (45, 45)
    K_test = K[np.ix_(test_idx, train_idx)]      # (1, 45) — rows=test, cols
    krr = KernelRidge(kernel="precomputed", alpha=1.0)
    krr.fit(K_train, y[train_idx])
    y_pred[test_idx] = krr.predict(K_test)
```

The `np.ix_()` **indexing** is critical for correctly sub-selecting the kernel matrix. The test-set kernel matrix must have shape `(n_test, n_train)`, not `(n_test, n_test)`. scikit-learn

Mantel test via scikit-bio

```python
from skbio.stats.distance import DistanceMatrix, mantel

dm1 = DistanceMatrix(D_brain)     # must be symmetric, hollow (zero diagonal
dm2 = DistanceMatrix(D_clinical)

coeff, p_value, n = mantel(
    dm1, dm2,
    method='spearman',       # preferred for neuroimaging (robust to nonline
    permutations=9999,       # default is only 999; increase for publicatior
    alternative='two-sided'
)
```

Mantel test accepts **only symmetric distance matrices**—no rectangular or asymmetric inputs. Scikit Scikit scikit-bio v0.7.2 (Feb 2026) supports Python 3.10–3.14; PyPI install via `conda install -c conda-forge scikit-bio` for reliability. Scikit

Integration: hemisphere mirroring, caching, and parallelism

Hemisphere mirroring requires three operations, not just one. Negating the x-coordinate is necessary but insufficient— Three.js you must also reverse face winding and re-estimate normals:

```python
rh_coords_mirrored = rh_coords.copy()
rh_coords_mirrored[:, 0] *= -1              # 1. Reflect across midsagittal
rh_faces_mirrored = rh_faces[:, [0, 2, 1]]  # 2. Reverse winding order (swap
# 3. Normals will be recomputed correctly by robust_laplacian or Open3D
```

Omitting the face-winding reversal causes the mesh to appear inside-out, github producing incorrect Laplacian weights. Carnegie Mellon University For point clouds without faces, reflection of x is sufficient—normals will be re-estimated from scratch by `robust_laplacian.point_cloud_laplacian()` or `open3d.estimate_normals()`.

Eigendecomposition caching for 439 subjects at 100 eigenvectors × 150k vertices requires ~**52 GB** in float64 (halved to ~**26 GB** in float32 with negligible precision loss). Use individual `.npy` files with memory-mapped reads: Tdeboissiere

```python
# Save
np.save(f'cache/{sid}/eigenvalues_k100.npy', evals)
np.save(f'cache/{sid}/eigenvectors_k100.npy', evecs.astype(np.float32))

# Load with memory mapping (fastest possible I/O)
evecs = np.load(f'cache/{sid}/eigenvectors_k100.npy', mmap_mode='r')
```

This directory-per-subject layout avoids file-locking issues during parallel access and allows trivial sharding across storage.

Thread safety: use `joblib.Parallel(n_jobs=16, backend='loky')` exclusively. All core libraries (robust_laplacian's C++ backend, ARPACK inside `eigsh`, POT's OpenMP-enabled EMD) have thread-safety caveats. Process-based parallelism via `loky` Joblib sidesteps all GIL and internal-state conflicts.

Recommended four-step workflow with estimated timings on 16 cores:

1. **Per-subject LBO + eigendecomposition** (embarrassingly parallel): 60–120s per subject → ~**30–55 minutes** for 439 subjects
2. **Pairwise functional maps or SWD** (embarrassingly parallel): 0.1–2s per pair × 96k pairs → ~**1–3 hours**
3. **Distance matrix assembly**: 439 × 439 = ~1.5 MB—trivially small, fills during step 2
4. **Statistical tests** (MDMR, Dcorr, Mantel, KRR): minutes each with 10,000 permutations

Total estimated wall time: 2–5 hours on a 16-core workstation. For the eigendecomposition step, use `sigma=1e-8` in `eigsh` (shift-invert mode) and set `v0=np.ones(n)` for reproducibility.

Conclusion

Three architectural decisions simplify the implementation significantly. First, **pyFM natively supports point clouds** by delegating to robust_laplacian internally— github GitHub the Poisson reconstruction step that many functional-map tutorials describe is unnecessary when starting from FreeSurfer surfaces. Second, **POT's sliced Wasserstein distance is the only OT method viable for full pairwise distance matrices** at this scale; GeomLoss is best reserved for differentiable optimization workflows, and FUGW for individual subject-pair alignment. Third, the spectral and transport paths produce complementary distance matrices that can both feed into the same downstream statistics module—MDMR for covariate-controlled group comparisons, distance correlation for association testing, and the SW kernel for prediction via kernel ridge regression.

The key version-sensitivity risks are FUGW's unstable API (under active development, no stable release guarantee), Open3D's historically slow Python version support (verify wheel availability for your exact Python minor version), and scikit-bio's occasional installation difficulties on pip (prefer conda). Pin all dependencies in a lockfile and test the complete environment before committing.